Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6600 - Principios de Sistemas Operativos

II Proyecto Programado

RAMpage: Simulación de algoritmos de paginación

Valerie M. Hernández Fernández
valeriehernandez@estudiantec.cr
2020010829

San José, Costa Rica
Octubre 2026

Índice general

1. Introducción **2**
- 1.1. Gestión de memoria en los sistemas operativos . . . 2
- 1.2. Paginación y memoria virtual . . . 3

2. Desarrollo **4**
- 2.1. El simulador RAMpage . . . 4
- 2.2. Algoritmo óptimo (OPT) . . . 7
  - 2.2.1. Descripción . . . 7
  - 2.2.2. Implementación . . . 8
- 2.3. FIFO y Second Chance . . . 9
  - 2.3.1. Descripción de FIFO . . . 9
  - 2.3.2. Implementación de FIFO . . . 9
  - 2.3.3. Descripción de Second Chance . . . 9
  - 2.3.4. Implementación de Second Chance . . . 11
- 2.4. LRU . . . 13
  - 2.4.1. Descripción . . . 13
  - 2.4.2. Implementación . . . 13
- 2.5. LFU . . . 14
  - 2.5.1. Descripción . . . 14
  - 2.5.2. Implementación . . . 15

3. Instalación y uso **17**
- 3.1. Requisitos . . . 17
- 3.2. Instalación . . . 17
- 3.3. Ejecución . . . 19
- 3.4. Configuración de la simulación . . . 19
- 3.5. Simulación . . . 20
- 3.6. Formato del archivo de operaciones . . . 22
- 3.7. Pruebas automáticas (opcional) . . . 22

4. Conclusiones **24**

Referencias **28**

Capítulo 1

Introducción

Este informe documenta RAMpage, un simulador que compara el algoritmo óptimo de reemplazo de páginas con los algoritmos FIFO, Second Chance, LRU y LFU. El simulador modela una computadora con un núcleo, una instrucción por segundo, 400 KB de memoria RAM divididos en 100 páginas de 4 KB y una memoria virtual en disco sin límite, en la que cada acceso a disco toma 5 segundos. Antes de describir la implementación, esta introducción resume los conceptos de sistemas operativos en los que se basa.

1.1. Gestión de memoria en los sistemas operativos

Varios programas comparten la misma memoria física, y el sistema operativo debe repartirla entre ellos sin que se interfieran. Para lograrlo, el sistema operativo ofrece a cada programa una abstracción llamada espacio de direcciones, que es la visión que tiene el programa en ejecución de la memoria del sistema (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 13). Cada programa se comporta como si tuviera la memoria para sí mismo, aunque en realidad comparte la memoria física con los demás.

Esta virtualización de la memoria persigue tres objetivos (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 13):

- **Transparencia:** el programa no debe notar que su memoria está virtualizada.
- **Eficiencia:** la virtualización debe costar lo menos posible en tiempo y en espacio.
- **Protección:** el sistema operativo debe aislar a los procesos entre sí y protegerse a sí mismo de ellos.

1.2. Paginación y memoria virtual

La paginación es un esquema de manejo de memoria que permite que el espacio de direcciones físico de un proceso no sea contiguo (Silberschatz, Galvin, y Gagne, 2018, p. 360). Divide la memoria lógica en bloques de tamaño fijo llamados páginas, y la memoria física en bloques del mismo tamaño llamados marcos (Silberschatz y cols., 2018, p. 360). Cada proceso tiene una tabla de páginas que indica, para cada página virtual, si está en un marco de la memoria RAM o si reside en disco (Kerrisk, 2010, p. 119). Como cualquier marco libre puede asignarse a cualquier página, la paginación no produce fragmentación externa; sí puede producir fragmentación interna, porque los marcos se asignan completos y, si la memoria que pide un proceso no coincide con los límites de las páginas, el último marco queda parcialmente vacío (Silberschatz y cols., 2018, p. 362). En RAMpage, por ejemplo, una solicitud de 500 bytes ocupa una página completa de 4096 bytes y desperdicia 3596 bytes.

La memoria virtual separa la memoria lógica que percibe el programador de la memoria física, lo que permite ofrecer una memoria virtual muy grande aunque solo se disponga de una memoria física menor (Silberschatz y cols., 2018, p. 390). Esto es posible gracias a la localidad de referencia: los programas tienden a acceder a direcciones cercanas a las que usaron recientemente (localidad espacial) y a volver a usar en el futuro cercano las mismas direcciones (localidad temporal), por lo que se puede ejecutar un programa con solo una parte de su espacio de direcciones en RAM (Kerrisk, 2010, pp. 118-119). Las páginas que no están en uso se guardan en el área de intercambio o *swap*, un espacio reservado en disco, y se cargan en memoria solo cuando se necesitan (Kerrisk, 2010, p. 119). Cada entrada de la tabla de páginas tiene un bit de presencia que indica si la página está en memoria o en disco (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 21). Cuando un proceso accede a una página que no está en memoria física ocurre un fallo de página: el núcleo suspende la ejecución del proceso mientras trae la página desde el disco (Kerrisk, 2010, p. 119).

Si la memoria física está llena cuando ocurre un fallo, el sistema operativo debe elegir una página para enviar a disco y liberar su marco. Esa decisión es la política de reemplazo, y de ella depende cuántos fallos de página ocurren (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 22). Una mala elección no produce resultados incorrectos, pero aumenta la tasa de fallos y hace más lenta la ejecución (Silberschatz y cols., 2018, p. 405). Cuando un proceso no tiene suficientes marcos, reemplaza páginas que vuelve a necesitar de inmediato y falla una y otra vez; esta actividad de paginación excesiva se llama *thrashing*, y se dice que un proceso la sufre cuando pasa más tiempo paginando que ejecutando (Silberschatz y cols., 2018, p. 419). Los algoritmos de reemplazo que compara este proyecto son las distintas respuestas a la pregunta de qué página sacar.

Capítulo 2

Desarrollo

2.1. El simulador RAMpage

RAMpage es una aplicación web escrita en TypeScript con React y Vite. El núcleo del simulador está separado de la interfaz: la unidad de manejo de memoria (MMU), los cinco algoritmos, el generador de operaciones y el lector de archivos son código TypeScript sin dependencias de React, cubierto por 74 pruebas automáticas.

Los procesos ejecutan cuatro operaciones:

- `new(pid, size)`: reserva `size` bytes para el proceso y devuelve un puntero lógico. Los punteros se numeran por orden de aparición y nunca se reutilizan.
- `use(ptr)`: accede a todas las páginas del puntero.
- `delete(ptr)`: libera el puntero y sus páginas.
- `kill(pid)`: libera toda la memoria del proceso.

El reloj de la simulación suma 1 segundo por cada página en memoria (acierto o *hit*) y 5 segundos por cada página que hay que traer de disco (fallo), que también se suman al tiempo de *thrashing*. Toda página nueva cuenta como fallo. Las operaciones `delete` y `kill` no acceden a páginas, por lo que no suman tiempo.

La MMU actualiza los datos de cada página de la misma forma para todos los algoritmos: el momento en que se cargó, el último acceso, la cantidad de accesos y el bit de referencia. Cada algoritmo solo decide qué página sale de memoria cuando se necesita un marco. Durante una operación, las páginas del puntero en uso quedan fijas en memoria, para que una operación nunca desaloje sus propias páginas. Si hay marcos libres, se usa el de menor número, de modo que la misma secuencia produce siempre el mismo resultado.

En la pantalla de configuración (figura 2.1) el usuario elige el algoritmo que se compara contra OPT y genera las operaciones a partir de una semilla, P

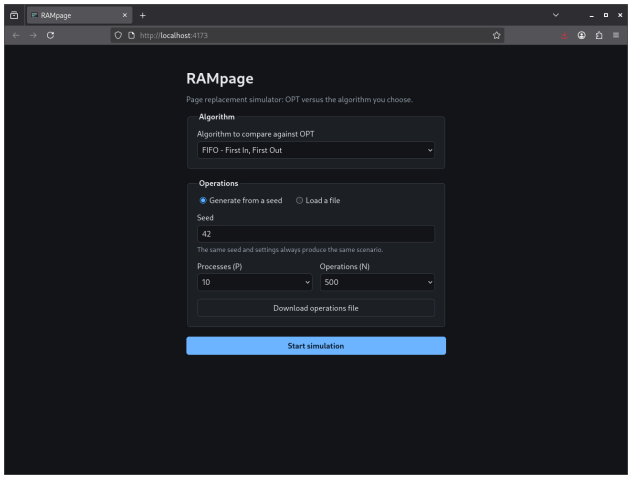


Figura 2.1: Pantalla de configuración: algoritmo, semilla, cantidad de procesos (P) y de operaciones (N)

y N, o carga un archivo de operaciones. La misma semilla genera siempre el mismo escenario, y la secuencia generada se puede descargar antes de simular. Los archivos se validan al cargarlos, e indican la línea u operación del primer error (figuras 2.2 y 2.3).

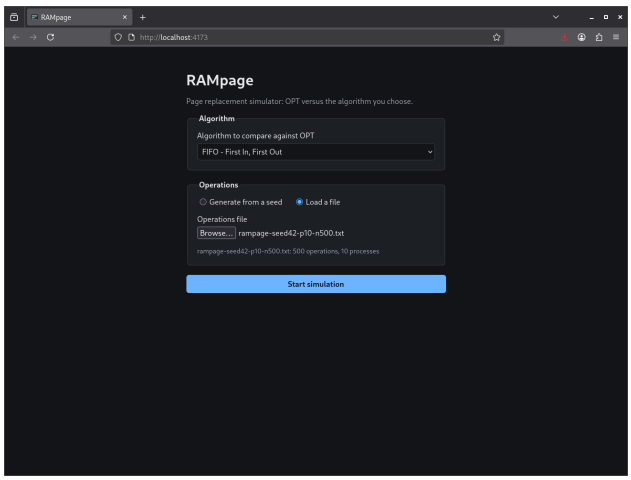


Figura 2.2: Archivo de operaciones cargado y validado

La pantalla de simulación ejecuta OPT y el algoritmo elegido sobre la misma secuencia: en cada paso, ambos ejecutan la misma operación, por lo que avanzan juntos y terminan al mismo tiempo. Para cada uno muestra los 100 marcos de

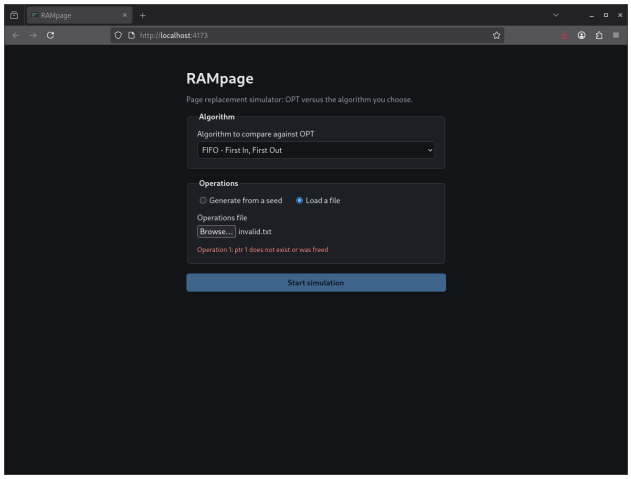


Figura 2.3: Archivo rechazado: usa un puntero que no existe

RAM coloreados por proceso, la tabla de páginas de la MMU y las métricas pedidas. Las páginas usadas por la última operación aparecen remarcadas con ▷, y las enviadas a disco, en rojo con ↓. La columna MARK muestra el dato que usa cada algoritmo para decidir.

Para verificar el modelo se calculó a mano el ejemplo del Anexo 1 del enunciado (tabla 2.1). Como solo usa 4 páginas, la memoria nunca se llena y todos los algoritmos deben dar el mismo resultado. La figura 2.4 muestra que el simulador reproduce exactamente el cálculo: 28 segundos, 8 aciertos y 4 fallos.

#	Operación	Resultado	Reloj (s)
1	`new(1,500)` → ptr 1	fallo	5
2	`use(1)`	acierto	6
3	`new(1,1000)` → ptr 2	fallo	11
4	`use(1)`	acierto	12
5	`use(2)`	acierto	13
6	`new(2,500)` → ptr 3	fallo	18
7	`use(3)`	acierto	19
8	`use(1)`	acierto	20
9	`new(2,50)` → ptr 4	fallo	25
10	`use(4)`	acierto	26
11	`delete(1)`	-	26
12	`use(2)`	acierto	27
13	`use(3)`	acierto	28
14 a 16	`delete(2)`, `kill(1)`, `kill(2)`	-	28

Cuadro 2.1: Cálculo a mano del Anexo 1

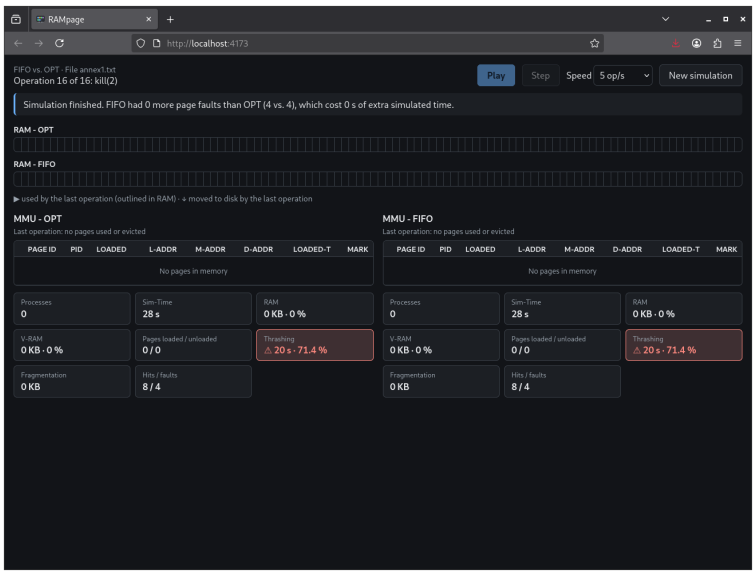


Figura 2.4: El Anexo 1 en el simulador: 28 s, 8 aciertos y 4 fallos, igual al cálculo a mano

Todas las capturas de este capítulo usan el mismo escenario: semilla 42, 10 procesos y 500 operaciones. Para comparar los algoritmos ante la misma situación, se muestra cada uno en la operación 163, `new(9,14350)`, que necesita 4 páginas nuevas con la memoria llena y obliga a cada algoritmo a elegir 4 víctimas.

2.2. Algoritmo óptimo (OPT)

2.2.1. Descripción

El algoritmo óptimo, conocido como OPT o MIN, reemplaza la página que no se va a usar durante el período más largo (Silberschatz y cols., 2018, p. 406). Garantiza la menor tasa de fallos posible para una cantidad fija de marcos y no sufre la anomalía de Belady (Silberschatz y cols., 2018, p. 406). Sin embargo, es difícil de implementar porque requiere conocer de antemano la secuencia de referencias futuras, por lo que se usa principalmente en estudios de comparación (Silberschatz y cols., 2018, p. 407): indica cuánto le falta a un algoritmo real para ser perfecto.

Ni siquiera OPT evita todos los fallos. La primera referencia a cada página produce un fallo que todos los algoritmos deben sufrir (Silberschatz y cols., 2018, p. 407). En RAMpage esto corresponde a la regla del enunciado de contar toda página nueva como fallo, y explica por qué OPT también supera el 50 %

de *thrashing* en las simulaciones.

2.2.2. Implementación

En el simulador, OPT sí es posible porque la secuencia completa existe antes de empezar. Al crearse, el algoritmo recorre la secuencia y guarda, para cada puntero, la lista ordenada de las operaciones que lo usan. Para elegir una víctima busca, por búsqueda binaria, el próximo uso de cada página candidata posterior a la operación actual, y desaloja la de uso más lejano. Una página que no se vuelve a usar tiene próximo uso infinito y es la primera candidata. Si hay empate, sale la página que se cargó primero. La columna MARK muestra el número de la próxima operación que usa la página, o ∞.

Como verificación, una prueba automática ejecuta los cinco algoritmos sobre 27 secuencias generadas (las 9 combinaciones de P y N, con 3 semillas): en las 108 comparaciones, OPT nunca tuvo más fallos que otro algoritmo. Este resultado es evidencia empírica y no una demostración, porque el modelo del proyecto agrega punteros de varias páginas y el fijado de páginas durante una operación.

En la operación 187, `new(6,4460)` (figura 2.5), OPT desaloja las páginas 1 y 2. Ambas tienen MARK ∞: OPT eligió justo páginas que no se vuelven a usar.

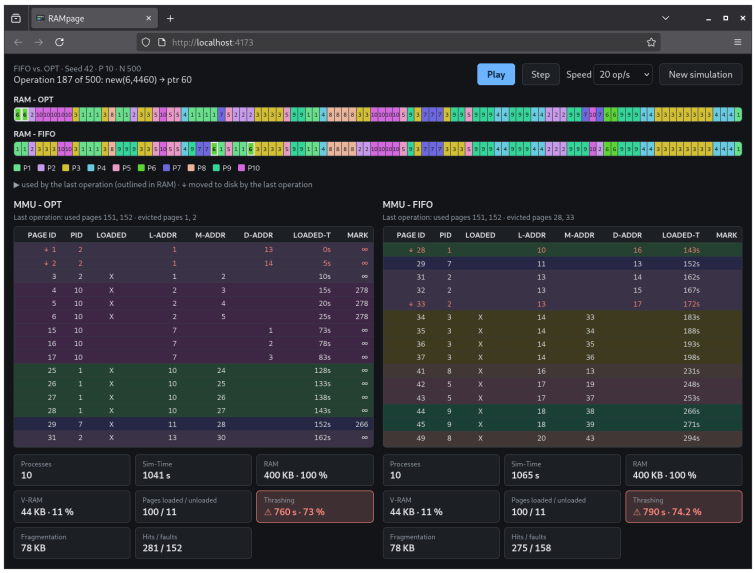


Figura 2.5: Operación 187: OPT desaloja las páginas 1 y 2, que no se vuelven a usar (∞)

2.3. FIFO y Second Chance

2.3.1. Descripción de FIFO

FIFO (*first in, first out*) asocia a cada página el momento en que entró a memoria y, cuando debe reemplazar una, elige la más antigua; se puede implementar con una cola en la que las páginas entran por el final y se reemplaza la del frente (Silberschatz y cols., 2018, pp. 404-405). Es fácil de entender y de programar, pero su rendimiento no siempre es bueno: la página reemplazada puede ser un módulo que ya no se necesita o una variable muy usada que se inicializó al principio (Silberschatz y cols., 2018, p. 405). Además, FIFO puede presentar la anomalía de Belady, en la que la tasa de fallos aumenta al aumentar la cantidad de marcos disponibles (Silberschatz y cols., 2018, p. 406).

2.3.2. Implementación de FIFO

Cada vez que una página entra a memoria recibe un número de secuencia único y creciente. FIFO desaloja la página candidata con el menor número de secuencia de carga. Como los números nunca se repiten, no hay empates. FIFO no usa ninguna marca, por lo que su columna MARK queda vacía.

En la operación 163 (figura 2.6), FIFO desaloja las páginas 15, 16, 17 y 25, las que llevan más tiempo en memoria (cargadas entre los segundos 73 y 128). OPT desaloja otras: 72, 73, 97 y 98. En esta operación, las víctimas de FIFO resultaron ser, por casualidad, páginas que tampoco se volvían a usar, por lo que ambos algoritmos tienen hasta aquí las mismas estadísticas. La operación muestra el criterio de cada uno, no un costo.

El costo aparece en la operación 180, `use(1)` (figura 2.7). El puntero 1 fue lo primero que se cargó, así que FIFO lo desalojó en cuanto necesitó espacio. OPT, en cambio, lo conservó porque sabía que la operación 180 lo iba a pedir. En esa operación, OPT obtiene 3 aciertos (3 s) y FIFO, 3 fallos (15 s): debe traer de vuelta las tres páginas, como muestra su columna LOADED-T (segundos 1004 a 1014, frente a 0 a 10 en OPT).

Al terminar la simulación (figura 2.8), FIFO acumula 188 fallos más que OPT, lo que cuesta 752 segundos adicionales.

2.3.3. Descripción de Second Chance

Second Chance corrige la debilidad de FIFO usando un bit de referencia. El hardware enciende el bit de una página cada vez que se hace referencia a ella, y el sistema operativo es el que lo apaga (Silberschatz y cols., 2018, p. 409). El algoritmo es básicamente FIFO: cuando selecciona una página, revisa su bit. Si vale 0, la reemplaza; si vale 1, le da una segunda oportunidad, apaga su bit, reinicia su tiempo de llegada al tiempo actual y pasa a la siguiente página de la cola (Silberschatz y cols., 2018, p. 410). Una forma de implementarlo es el algoritmo de reloj, que recorre las páginas en una cola circular (Silberschatz y cols., 2018, p. 410). En el peor caso, cuando todos los bits están encendidos, el

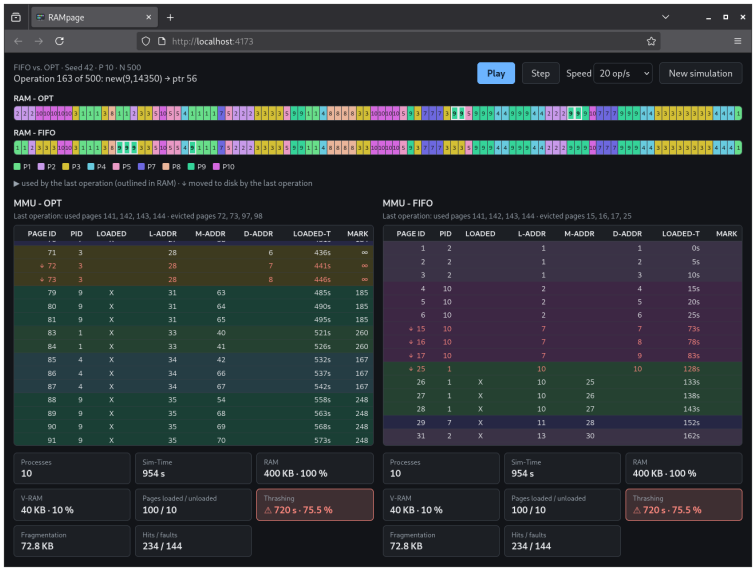


Figura 2.6: Operación 163: FIFO desaloja las páginas más antiguas; OPT, las que no se vuelven a usar

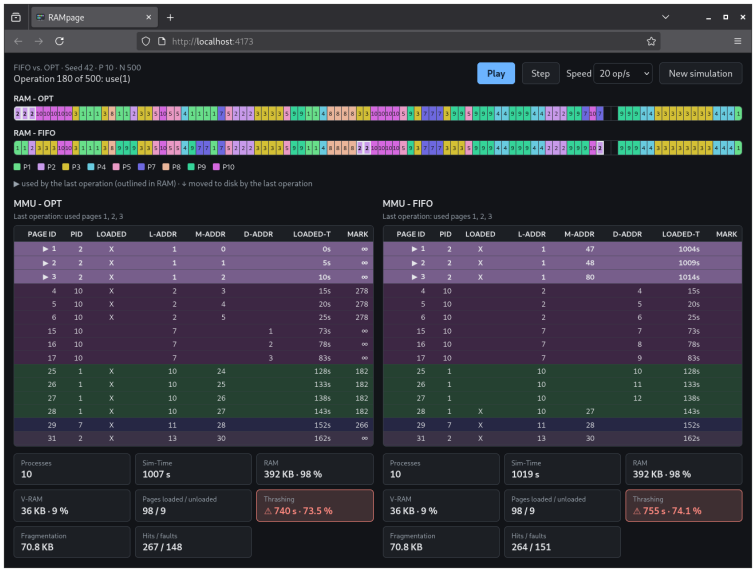


Figura 2.7: Operación 180: FIFO debe traer de disco el puntero 1, que OPT conservó en memoria

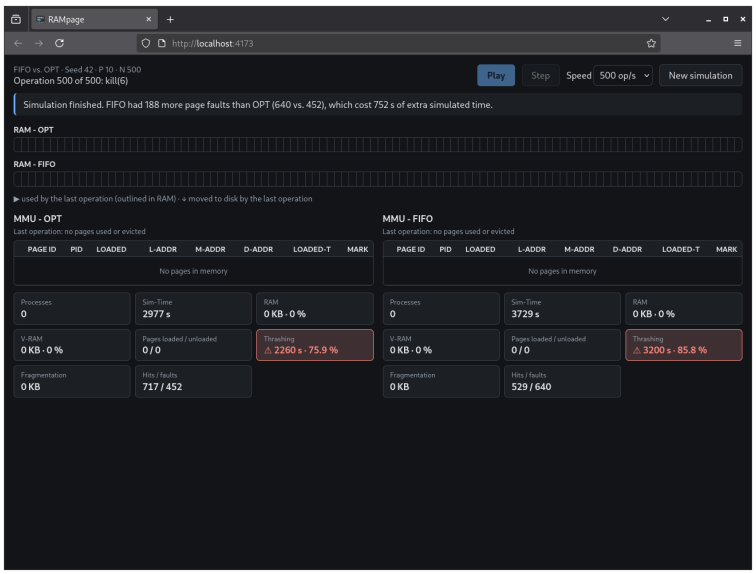


Figura 2.8: Resultado final de FIFO: 188 fallos más que OPT

algoritmo da una vuelta completa apagándolos y Second Chance degenera en FIFO (Silberschatz y cols., 2018, p. 410).

2.3.4. Implementación de Second Chance

La MMU enciende el bit de referencia cada vez que una página se carga o se accede, y lo apaga cuando la página sale a disco. Al elegir una víctima, el algoritmo toma la página con menor secuencia de carga. Si su bit está apagado, la desaloja. Si está encendido, lo apaga y le asigna un número de secuencia nuevo, lo que equivale a reiniciar su tiempo de llegada y moverla al final de la cola, tal como describe el algoritmo. El ciclo tiene un límite explícito: en el peor caso, todas las páginas tienen el bit encendido, la primera vuelta los apaga todos y la siguiente encuentra víctima. Mover una página al final de la cola no cambia su columna LOADED-T, que sigue mostrando cuándo entró realmente a memoria. La columna MARK muestra el bit de referencia (1 o 0).

En la operación 163 (figura 2.9), Second Chance elige las mismas víctimas que FIFO, porque las páginas más antiguas ya tenían el bit apagado de vueltas anteriores. La figura 2.10 muestra el mismo momento con la tabla desplazada hacia las páginas recientes, donde se ven bits encendidos y apagados: las víctimas siempre salen del grupo con bit 0.

Al final (figura 2.11), Second Chance tiene 179 fallos más que OPT: 9 menos que FIFO.

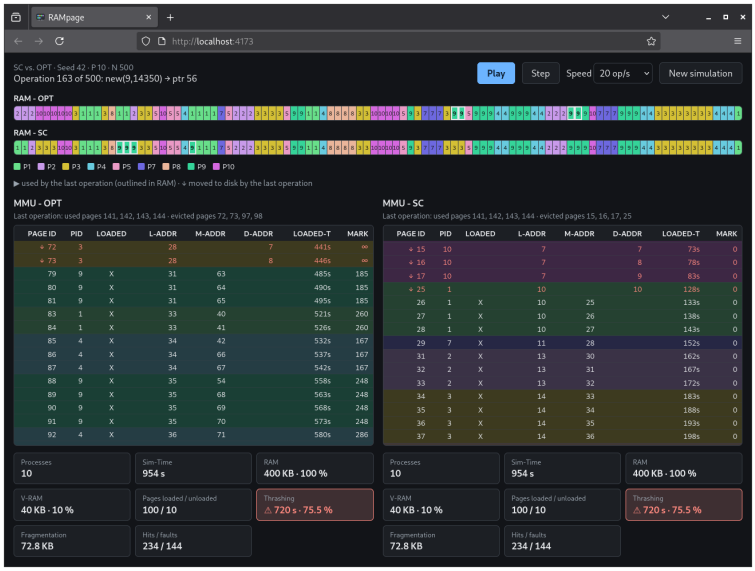


Figura 2.9: Operación 163 con Second Chance: las páginas más antiguas tienen el bit en 0

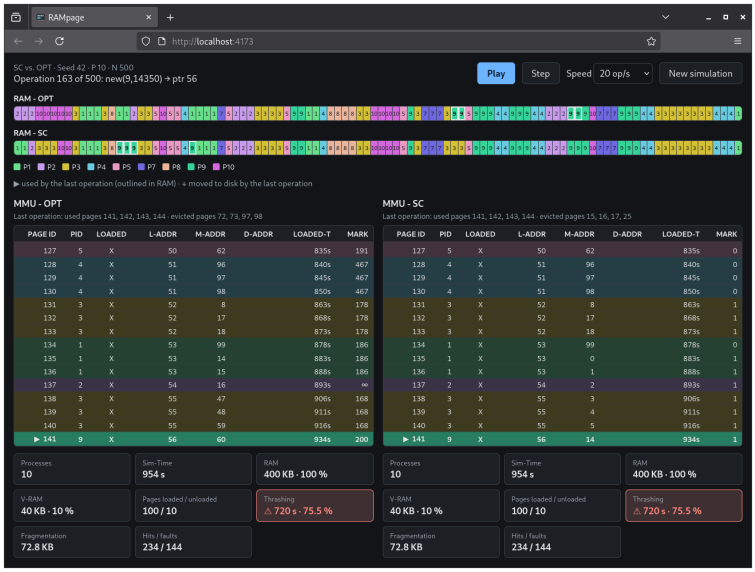


Figura 2.10: Bits de referencia de las páginas recientes en la operación 163

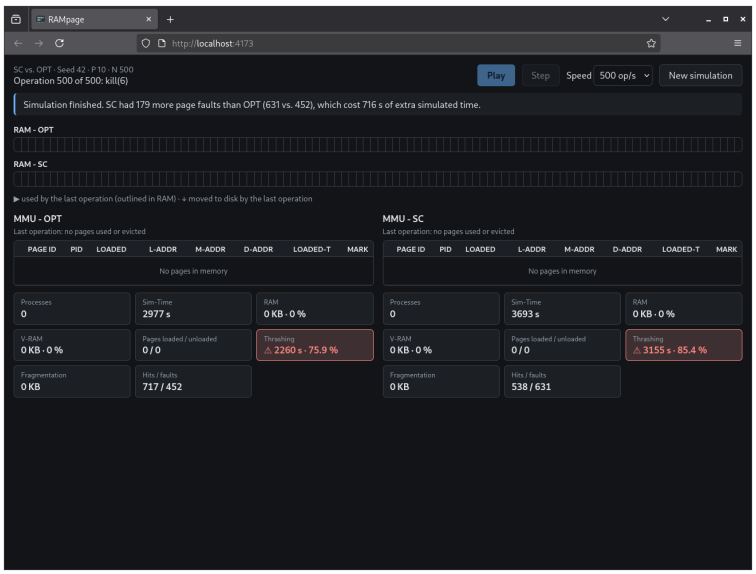


Figura 2.11: Resultado final de Second Chance: 179 fallos más que OPT

2.4. LRU

2.4.1. Descripción

LRU (*least recently used*) usa el pasado reciente como aproximación del futuro cercano: asocia a cada página el momento de su último uso y, cuando debe reemplazar una, elige la que lleva más tiempo sin usarse (Silberschatz y cols., 2018, p. 407). Puede verse como el algoritmo óptimo mirando hacia atrás en el tiempo en lugar de hacia adelante (Silberschatz y cols., 2018, p. 407). Es un algoritmo basado en el historial, que aprovecha el principio de localidad, y por eso puede superar a políticas que no guardan historial como FIFO (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 22). A diferencia de FIFO, LRU no sufre la anomalía de Belady (Arpaci-Dusseau y Arpaci-Dusseau, 2018, cap. 22).

2.4.2. Implementación

La implementación sigue el esquema de contadores: se agrega un reloj lógico que avanza con cada referencia a memoria, se copia su valor en un campo de tiempo de uso de la página cada vez que se hace referencia a ella, y se reemplaza la página con el menor valor (Silberschatz y cols., 2018, p. 408). En RAMpage, cada carga y cada acierto asignan a la página un número de secuencia de último acceso, único y creciente, y LRU desaloja la página candidata con el menor de estos números. Como no se repiten, no hay empates. La columna MARK muestra el número de secuencia del último acceso.

En la operación 163 (figura 2.12), LRU desaloja las páginas 71, 72, 73 y 44. Las visibles tienen MARK 157, 158 y 159, los valores más bajos de la tabla (el resto parte de 174): son las páginas usadas hace más tiempo.

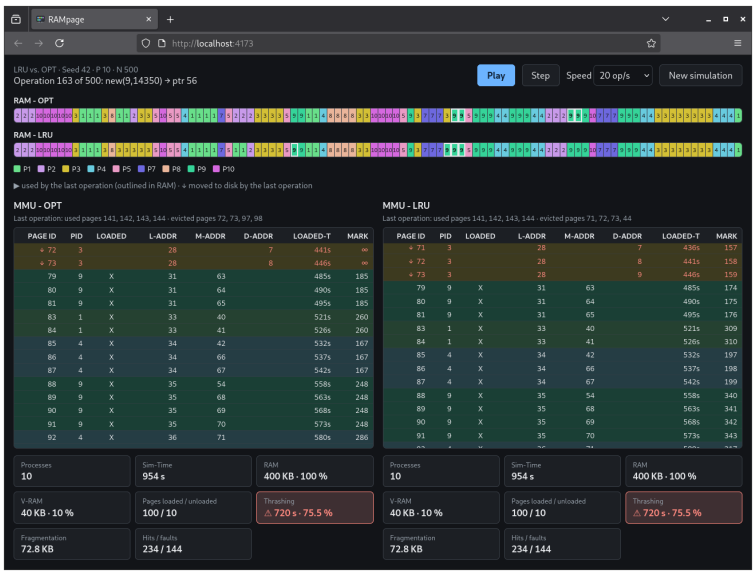


Figura 2.12: Operación 163 con LRU: las víctimas tienen los últimos accesos más antiguos

Al final (figura 2.13), LRU tiene 178 fallos más que OPT, el mejor resultado de los cuatro algoritmos reales.

2.5. LFU

2.5.1. Descripción

LFU (*least frequently used*) mantiene un contador de las referencias a cada página y reemplaza la que tiene el conteo más bajo, bajo la idea de que una página usada activamente debe tener un conteo alto (Silberschatz y cols., 2018, p. 411). Igual que LRU, se basa en el historial, pero mide la frecuencia de uso en lugar de qué tan reciente fue. Tiene dos problemas conocidos. Una página muy usada al inicio de un proceso conserva un conteo alto y permanece en memoria aunque ya no se necesite (Silberschatz y cols., 2018, pp. 411-412). Y, en sentido contrario, la página con el conteo más bajo probablemente acaba de cargarse y todavía no ha tenido oportunidad de usarse (Silberschatz y cols., 2018, p. 412). Por estas razones LFU es poco común: su implementación es costosa y no se aproxima bien a OPT (Silberschatz y cols., 2018, p. 412).

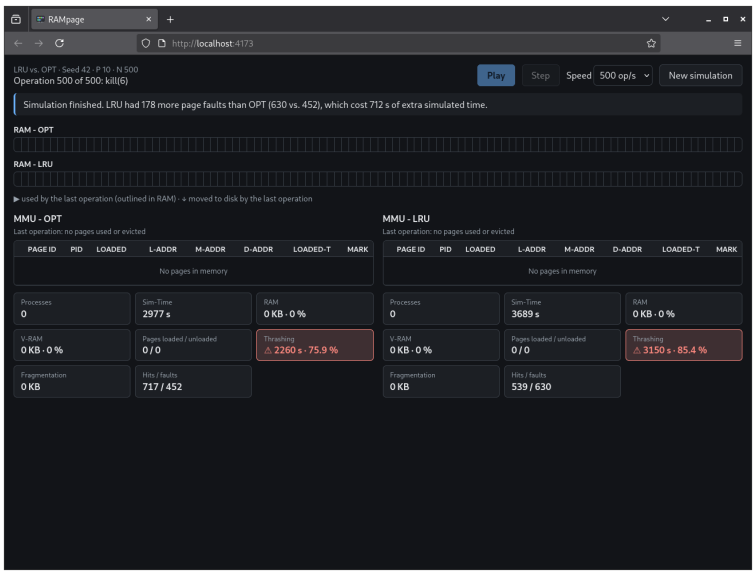


Figura 2.13: Resultado final de LRU: 178 fallos más que OPT

2.5.2. Implementación

La MMU cuenta los accesos de cada página. El contador vuelve a 1 cada vez que la página entra a memoria, de modo que mide los usos desde su última carga. LFU desaloja la página candidata con menos accesos. Muchas páginas comparten el mismo conteo (todas las recién cargadas tienen 1), así que los empates se resuelven desalojando la que se cargó primero. Tanto el reinicio del contador como el desempate son decisiones de diseño del proyecto. La columna MARK muestra la cantidad de accesos.

En la operación 163 (figura 2.14), LFU desaloja las páginas 79, 80, 81 y 85, todas con un solo acceso, y conserva las páginas 83 y 84, que tienen 5. Según la tabla de OPT en la misma figura, esas víctimas se iban a usar en las operaciones 185 y 167: LFU sacó páginas que se necesitaban pocas operaciones después. Es justamente el segundo problema descrito: una página recién cargada siempre tiene el conteo más bajo, así que es la primera en salir aunque esté a punto de usarse.

Al final (figura 2.15), LFU tiene 191 fallos más que OPT, el peor resultado de los cuatro.

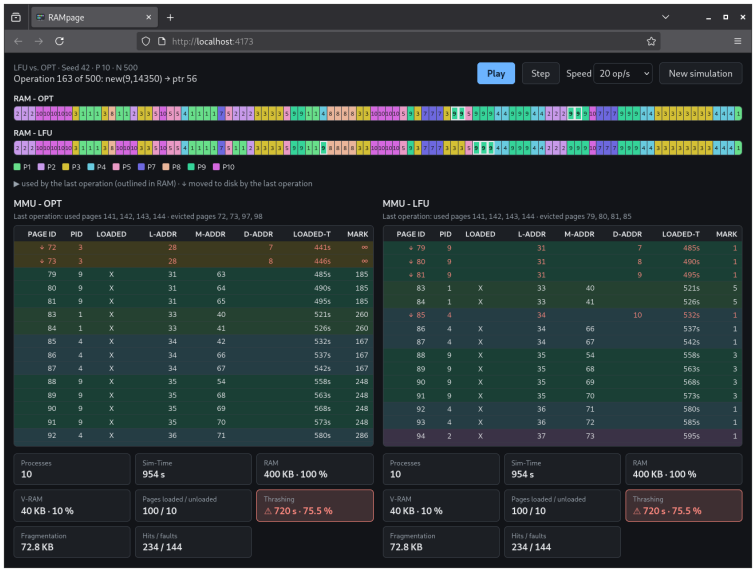


Figura 2.14: Operación 163 con LFU: desaloja páginas recién cargadas que OPT sabe que se usarán pronto

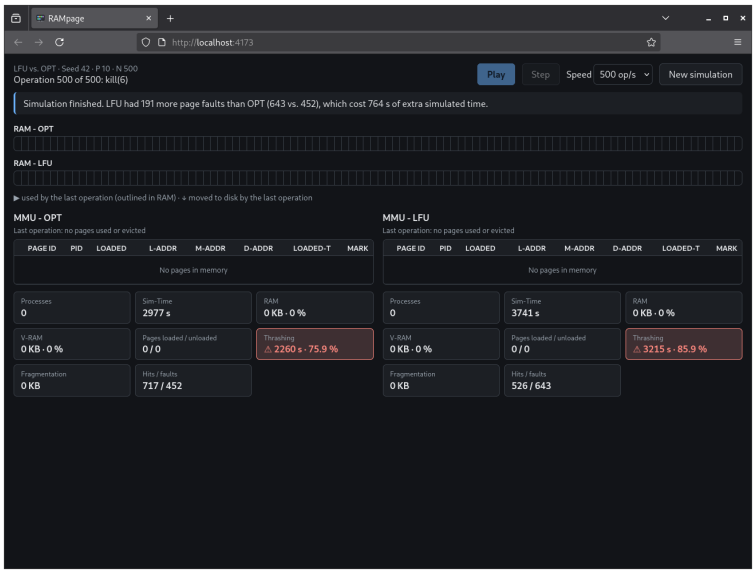


Figura 2.15: Resultado final de LFU: 191 fallos más que OPT

Capítulo 3

Instalación y uso

Este capítulo es la guía para instalar, ejecutar y usar RAMpage. Los pasos deben seguirse en el orden indicado. El código fuente está disponible en `https://github.com/valeriehernandez-7/RAMpage`.

Los comandos se escriben en una terminal, uno por línea, y se ejecutan con la tecla `Enter`. Los nombres de botones y campos de la aplicación se escriben en **negrita** tal como aparecen en pantalla; la interfaz está en inglés.

3.1. Requisitos

- Sistema operativo Debian 13 de 64 bits con escritorio. El proyecto se desarrolló y probó en Debian 13.7 con GNOME, en una máquina virtual de VirtualBox.
- Un usuario con permiso para usar `sudo`.
- Conexión a Internet durante la instalación, para descargar los paquetes de Debian, el código fuente y las dependencias del proyecto. Una vez instalado, el simulador funciona sin conexión.
- Un navegador web. El proyecto se probó en Firefox ESR 140, el navegador incluido en Debian 13.
- Node.js 20.19.0 o una versión más reciente, npm y Git. El paso 2 de la sección 3.2 los instala.

3.2. Instalación

La instalación se hace una sola vez.

1. **Abrir una terminal.** En GNOME, presionar la tecla `Super` (la tecla con el logotipo de Windows), escribir `Terminal` y presionar `Enter`. La terminal abre en la carpeta personal del usuario.

2. **Instalar Node.js, npm y Git.** Los tres se instalan desde los repositorios oficiales de Debian. En Debian, `nodejs` y `npm` son paquetes separados, por lo que se instalan los dos:

   ```
   sudo apt update
   sudo apt install -y nodejs npm git
   ```

   El comando `sudo` solicita la contraseña del usuario. Al escribirla no se muestran caracteres en pantalla; esto es normal.

3. **Verificar la versión de Node.js:**

   ```
   node --version
   ```

   Debe mostrar `v20.19.0` o un número mayor; Debian 13.7 instala la versión `v20.19.2`. Si la versión es menor, los pasos siguientes se detienen con un error, porque el proyecto exige esa versión mínima.

4. **Descargar el código fuente.** El primer comando crea la carpeta `RAMpage` dentro de la carpeta actual y descarga el proyecto en ella. El segundo entra a esa carpeta, desde la cual se ejecutan todos los comandos que siguen:

   ```
   git clone https://github.com/valeriehernandez-7/RAMpage.git
   cd RAMpage
   ```

5. **Instalar las dependencias del proyecto:**

   ```
   npm ci
   ```

   Este comando descarga las bibliotecas que usa el proyecto (React, Vite y TypeScript, entre otras) en la carpeta `node_modules`, con las versiones exactas registradas en el archivo `package-lock.json`. Así se instala exactamente lo mismo que se usó y probó durante el desarrollo. Al terminar, el mensaje final indica la cantidad de paquetes agregados y `found 0 vulnerabilities`.

6. **Compilar la aplicación:**

   ```
   npm run build
   ```

   Este comando revisa los tipos de TypeScript y genera la aplicación lista para usarse en la carpeta `dist`. La compilación termina correctamente cuando la última línea dice `built in` seguido del tiempo que tardó. Si aparece un error, la carpeta `dist` no se genera y el paso siguiente no funciona.

3.3. Ejecución

1. **Iniciar el servidor** desde la carpeta `RAMpage`:

   ```
   npm run preview
   ```

   El comando inicia un servidor web local, muestra la dirección `Local: http://localhost:4173/` y abre esa dirección en el navegador predeterminado. La terminal queda ocupada por el servidor mientras la aplicación esté en uso; no debe cerrarse.

2. **Abrir la aplicación.** Si el navegador no se abre por sí solo, abrirlo manualmente y escribir en la barra de direcciones la dirección que muestra la terminal en la línea `Local:`, normalmente `http://localhost:4173/`. Si el puerto 4173 ya está en uso, el servidor elige otro y lo indica en esa misma línea.

3. **Detener el servidor** al terminar: volver a la terminal y presionar `Ctrl+C`.

Para volver a usar el simulador más adelante no hace falta repetir la instalación. Basta con abrir una terminal y ejecutar:

```
cd RAMpage
npm run preview
```

La aplicación compilada debe abrirse siempre por medio del servidor. Abrir el archivo `dist/index.html` con doble clic muestra una página en blanco, porque el navegador bloquea los módulos de JavaScript cargados directamente desde el sistema de archivos.

3.4. Configuración de la simulación

Al abrir la aplicación se muestra la pantalla de configuración (figura 2.1). La simulación se prepara así:

1. **Elegir el algoritmo** que se compara contra OPT: FIFO, SC (Second Chance), LRU o LFU. OPT se ejecuta siempre; este campo elige el segundo algoritmo.

2. **Elegir el origen de las operaciones**: generarlas a partir de una semilla (**Generate from a seed**) o cargarlas desde un archivo (**Load a file**). La pantalla muestra solo los campos del origen elegido.

3. **Si se generan a partir de una semilla:**

   a) Escribir la semilla, un número entero. Si el valor no es válido, el campo se marca en rojo con un mensaje que explica el error. La misma semilla, con los mismos P y N, genera siempre la misma secuencia de operaciones.

b) Elegir la cantidad de procesos (P): 10, 50 o 100.

c) Elegir la cantidad de operaciones (N): 500, 1000 o 5000.

d) Opcionalmente, presionar **Download operations file** para guardar la secuencia generada antes de simular. El archivo se guarda en la carpeta de descargas del navegador, con un nombre que incluye la configuración, por ejemplo `rampage-seed42-p10-n500.txt`. Ese archivo se puede cargar después con **Load a file** para repetir exactamente el mismo escenario.

4. **Si se carga un archivo:** presionar el selector de archivo y elegir un archivo de texto con el formato descrito en la sección 3.6. El archivo se valida en cuanto se selecciona:
   - Si es válido, se muestra la cantidad de operaciones y de procesos que contiene, por ejemplo `500 operations, 10 processes` (figura 2.2).
   - Si no es válido, se muestra en rojo la línea u operación del primer error y su causa (figura 2.3). En ese caso hay que corregir el archivo y volver a seleccionarlo.
5. **Presionar Start simulation.** El botón se habilita solo cuando la configuración es válida. La pantalla de configuración se reemplaza por la pantalla de simulación.

3.5. Simulación

La pantalla de simulación (figura 2.4) muestra, de arriba hacia abajo:

- **Encabezado**: la configuración usada, por ejemplo `FIFO vs. OPT · Seed 42 · P 10 · N 500`, o el nombre del archivo cargado. Debajo, la operación actual, por ejemplo `Operation 96 of 500: new(9,15927)` → `ptr 35`.
- **Controles**: **Play**/**Pause**, **Step**, el selector de velocidad y **New simulation**.
- **Barras de RAM**: una para OPT y otra para el algoritmo elegido, con los 100 marcos de RAM. Cada marco ocupado muestra el PID de su proceso y se colorea según ese proceso; la leyenda indica el color de cada PID. Al pasar el puntero del ratón sobre un marco se muestra su número, el PID, la página y el puntero.
- **Dos columnas**, una para OPT y otra para el algoritmo elegido. Cada una contiene la tabla de páginas de la MMU y, debajo, las métricas.

La simulación se controla así:

1. **Elegir la velocidad**: 1, 5, 20, 100 o 500 operaciones por segundo. La velocidad inicial es 5, lo bastante lenta para seguir cada operación. Se puede cambiar en cualquier momento.

2. **Presionar Play** para ejecutar la simulación de forma continua. El botón cambia a **Pause**, que la detiene en la operación actual.

3. **Presionar Step**, con la simulación en pausa, para ejecutar una sola operación. Sirve para revisar con calma el efecto de cada operación. El botón está deshabilitado mientras la simulación corre.

4. Al ejecutar la última operación la simulación se detiene sola, el botón **Play** se deshabilita y aparece un resumen que compara el algoritmo elegido contra OPT.

5. **Presionar New simulation** para volver a la pantalla de configuración. Si la simulación no ha terminado, la aplicación pide confirmación antes de salir.

En cada paso, OPT y el algoritmo elegido ejecutan la misma operación, por lo que siempre muestran el mismo punto de la secuencia. Las páginas usadas por la última operación aparecen remarcadas en las barras de RAM y con ▷ en las tablas. Las páginas enviadas a disco por esa operación aparecen en rojo con ↓. Debajo del título de cada tabla, una línea resume las páginas usadas y desalojadas, por ejemplo `Last operation: used pages 141, 142, 143, 144 · evicted pages 72, 73, 97, 98`. Con la simulación en pausa, cada tabla se desplaza hasta la primera página usada; las tablas también se pueden desplazar con el ratón o con las flechas del teclado.

Las columnas de la tabla de páginas son las del Anexo 2 del enunciado:

- `PAGE ID`: número de la página.
- `PID`: proceso dueño de la página.
- `LOADED`: `X` si la página está en RAM; vacío si está en disco.
- `L-ADDR`: puntero lógico al que pertenece la página.
- `M-ADDR`: marco de RAM que ocupa, si está en RAM.
- `D-ADDR`: posición en disco, si está en disco.
- `LOADED-T`: segundo de la simulación en que la página se cargó en RAM.
- `MARK`: el dato que usa el algoritmo para elegir víctima. En OPT es la próxima operación que usa la página (∞ si no se vuelve a usar); en SC, el bit de referencia; en LRU, la secuencia del último acceso; en LFU, la cantidad de accesos. FIFO no usa marca.

Las métricas de cada algoritmo son:

- **Processes**: procesos en ejecución.
- **Sim-Time**: tiempo simulado, en segundos.
- **RAM**: memoria real usada, en KB y en porcentaje.
- **V-RAM**: memoria virtual usada, en KB y como porcentaje de la RAM.
- **Pages loaded / unloaded**: páginas en RAM y en disco.
- **Thrashing**: tiempo dedicado a resolver fallos de página, en segundos y como porcentaje del tiempo simulado. Se muestra en rojo con un ícono de advertencia cuando supera el 50 %.
- **Fragmentation**: fragmentación interna, en KB.
- **Hits / faults**: aciertos y fallos de página.

3.6. Formato del archivo de operaciones

El archivo de operaciones es un archivo de texto plano con el formato del Anexo 1 del enunciado: una instrucción por línea. Los punteros se numeran según el orden en que aparecen las instrucciones `new`, empezando en 1. Por ejemplo:

```
new(1,500)
use(1)
new(1,1000)
use(1)
use(2)
delete(1)
kill(1)
```

Las instrucciones válidas son `new(pid,size)`, `use(ptr)`, `delete(ptr)` y `kill(pid)`, descritas en la sección 2.1. Además, el simulador acepta instrucciones separadas por comas o punto y coma, finales de línea de Windows, líneas vacías, espacios adicionales y mayúsculas. La figura 3.1 muestra un archivo con finales de línea de Windows que reproduce exactamente el resultado de la semilla 42.

3.7. Pruebas automáticas (opcional)

Desde la carpeta `RAMpage`, el siguiente comando ejecuta las 74 pruebas automáticas del núcleo del simulador:

```
npm test
```

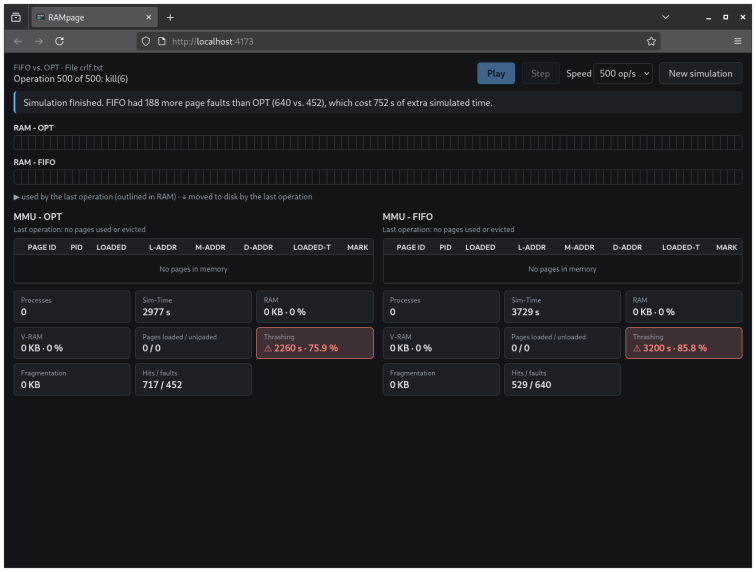


Figura 3.1: Archivo con finales de línea de Windows: mismo resultado que la semilla 42

Entre ellas están el cálculo a mano del Anexo 1 (28 segundos, 8 aciertos y 4 fallos) y la comprobación de que OPT nunca tiene más fallos de página que los demás algoritmos en las secuencias generadas. Al final, el resumen debe mostrar las líneas `# pass 74` y `# fail 0`.

Capítulo 4

Conclusiones

Para comparar los algoritmos se ejecutaron dos escenarios con la semilla 42: uno pequeño, con 10 procesos y 500 operaciones (tabla 4.1), y uno grande, con 100 procesos y 5000 operaciones (tabla 4.2 y figuras 4.1 a 4.4).

Algoritmo	Tiempo (s)	Aciertos / fallos	Thrashing	Fallos vs. OPT
OPT	2977	717 / 452	75.9 %	-
LRU	3689	539 / 630	85.4 %	+178
Second Chance	3693	538 / 631	85.4 %	+179
FIFO	3729	529 / 640	85.8 %	+188
LFU	3741	526 / 643	85.9 %	+191

Cuadro 4.1: Semilla 42, 10 procesos, 500 operaciones

Algoritmo	Tiempo (s)	Aciertos / fallos	Thrashing	Fallos vs. OPT
OPT	40760	3420 / 7468	91.6 %	-
LRU	50848	898 / 9990	98.2 %	+2522
Second Chance	50860	895 / 9993	98.2 %	+2525
FIFO	50860	895 / 9993	98.2 %	+2525
LFU	50972	867 / 10021	98.3 %	+2553

Cuadro 4.2: Semilla 42, 100 procesos, 5000 operaciones

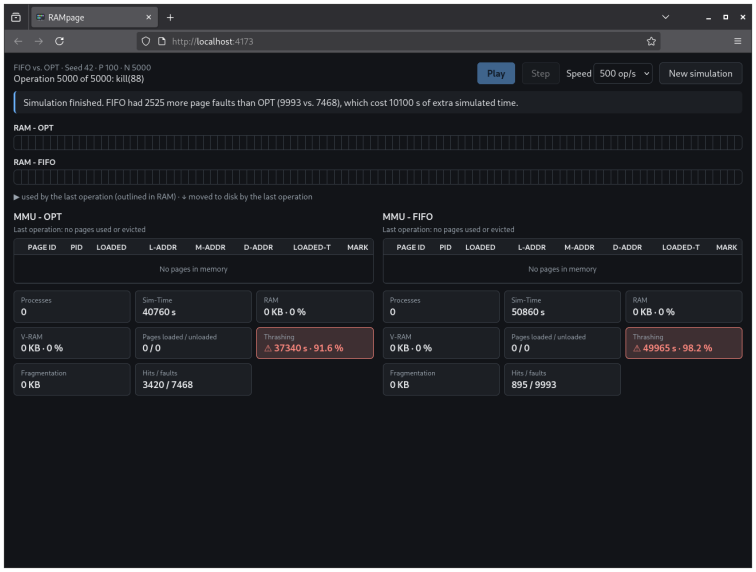


Figura 4.1: FIFO con 100 procesos y 5000 operaciones

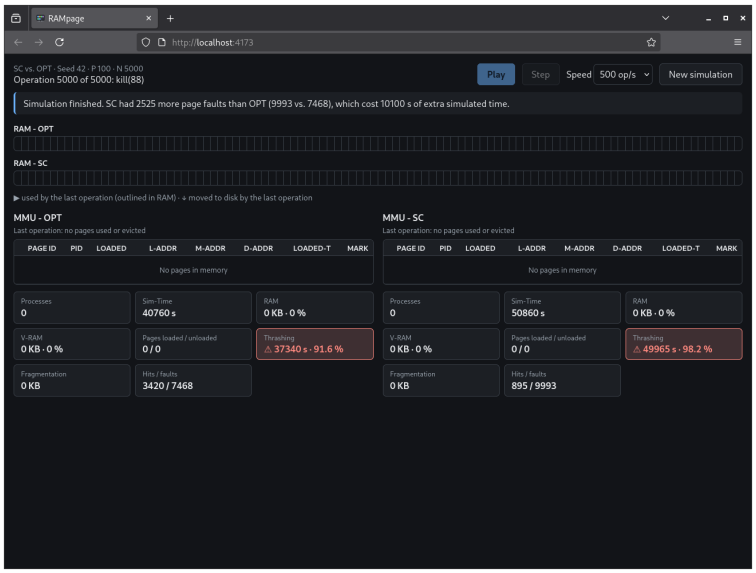


Figura 4.2: Second Chance con 100 procesos y 5000 operaciones: idéntico a FIFO

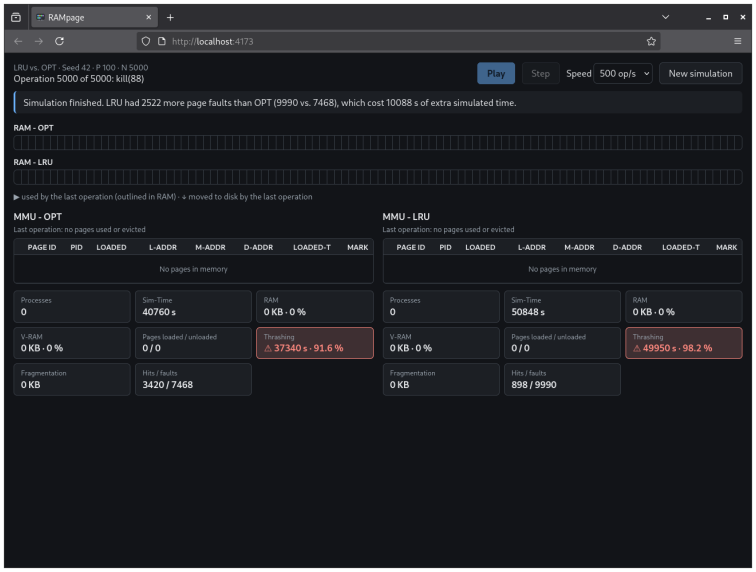


Figura 4.3: LRU con 100 procesos y 5000 operaciones

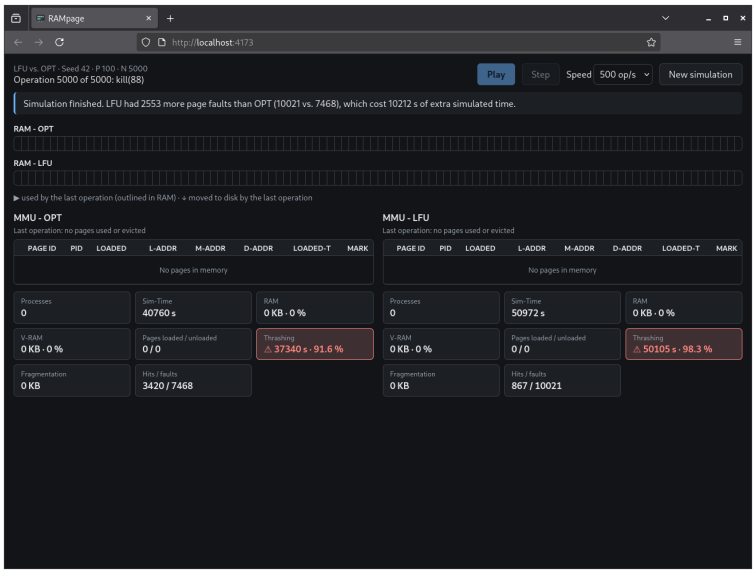


Figura 4.4: LFU con 100 procesos y 5000 operaciones

1. **OPT es una cota que ningún algoritmo real alcanza.** En ambos escenarios, los cuatro algoritmos reales tuvieron entre 34 % y 42 % más fallos que OPT. La diferencia se explica porque solo OPT conoce el futuro: en la operación 180, FIFO tuvo que traer de disco el puntero 1, que OPT había conservado porque sabía que se iba a usar.

2. **Las diferencias entre los algoritmos reales son pequeñas frente a la distancia con OPT.** El orden se repitió en ambos escenarios: LRU, luego Second Chance y FIFO, y por último LFU. Pero entre el mejor y el peor hubo apenas 13 fallos en el escenario pequeño y 31 en el grande (menos del 2 %), mientras que la distancia con OPT fue entre 13 y 80 veces mayor.

3. **LFU fue el peor algoritmo en ambos escenarios.** Una página recién cargada siempre tiene el conteo más bajo, así que LFU la desaloja primero aunque esté a punto de usarse. En la operación 163 sacó páginas que OPT sabía que se iban a usar en las operaciones 167 y 185. El resultado coincide con lo que señala la literatura: LFU no se aproxima bien a OPT (Silberschatz y cols., 2018, p. 412).

4. **El bit de referencia de Second Chance solo ayuda cuando hay localidad.** Con 10 procesos, Second Chance tuvo 9 fallos menos que FIFO. Con 100 procesos compitiendo por 100 marcos, una página casi nunca se vuelve a usar antes de llegar al frente de la cola, su bit no aporta información, y Second Chance dio exactamente el mismo resultado que FIFO. Es el comportamiento que predice la teoría: sin páginas que se distingan por su uso reciente, Second Chance degenera en FIFO (Silberschatz y cols., 2018, p. 410).

5. **El thrashing aumenta con la cantidad de procesos.** Con 10 procesos, el thrashing de los algoritmos reales rondó el 85 % del tiempo; con 100, el 98 %. Cuando los procesos no tienen suficientes marcos, reemplazan páginas que vuelven a necesitar de inmediato (Silberschatz y cols., 2018, p. 419). Con la memoria fija en 100 marcos, la cantidad de procesos que compiten por ella pesa más en el rendimiento que el algoritmo de reemplazo elegido.

Referencias

Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces*. Arpaci-Dusseau Books. Descargado de `https://pages.cs.wisc.edu/~remzi/OSTEP/`

Kerrisk, M. (2010). *The linux programming interface*. No Starch Press.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10ma ed.). Wiley.